Khoa Vu

Dr. Hossein Sayadi

CECS 341

10 May 2024

Lab Report

Introduction:

Generally speaking, Lab 3 promotes using the Vivado tool and IP Integrator to complete all the given tasks and instructions. Objectively, this lab aims to design the Arithmetic and Logic Unit (ALU) component using the behavioral VHDL model. Then, the designs should be converted into a block design using the IP integrator tool which can be verified via a testbench that runs simulation results and generated waveforms. All in all, the data below showcases the comprehensive lab report describing the steps in developing my design for Lab 3.

Lab Design:

To start off, the given ALU is consisted of control signals and two 32-bit inputs A and B; furthermore, the result of the operation appears on 32-bit output port called ALUout. The ALU also computes three flags bits Zero, Carryout, and Overflow which utilized the functions of AND, OR, XOR, ADD, SUBTRACT, and NOR gates. That being said, I wrote a VHDL code to design the ALU component that contained the entity, signals, and architecture codes as shown below.

ALU.vhd

Z:/Khoa Vu - Lab3/Khoa Vu - Lab3.srcs/sources_1/new/ALU.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use ieee.NUMERIC_STD.ALL;

entity ALU is
    port (A, B      : in STD_LOGIC_VECTOR (31 downto 0);
          ALUCntl   : in STD_LOGIC_VECTOR (3 downto 0);
          ALUout    : out STD_LOGIC_VECTOR (31 downto 0);
          Carryin   : in STD_LOGIC;
          Zero      : out STD_LOGIC;
          Cout      : out STD_LOGIC;
          Overflow  : out STD_LOGIC
          );

end ALU;

architecture Behavioral of ALU is
    signal ALU_Result : STD_LOGIC_VECTOR (31 downto 0);
    signal a32, b32, c32 : std_logic_vector (32 downto 0):=(others=>'0');
    signal add_result, sub_result : std_logic_vector (32 downto 0);
    signal add_ov, sub_ov : std_logic;

begin
    a32 <= '0' & A;
    b32 <= '0' & B;
    c32(0) <= Carryin;

    add_result <= a32 + b32 + Carryin;
    sub_result <= a32 - b32;

    with ALUCntl select
        ALU_Result <= add_result (31 downto 0) when "0010",
                      sub_result (31 downto 0) when "0110",
                      A AND B when "0000",
                      A OR B when "0001",
                      A XOR B when "0011",
                      not (A OR B) when "1100",
                      (others => 'X') when others;

    Zero <= '1' when ALU_Result =x"00000000" else '0';

    -- Carryout flag
    Cout <= add_result(32) when ALUCntl = "0010" else
            sub_result(32) when ALUCntl = "0110" else
            'X';

    -- Overflow flag
    add_ov <= (A(31) and B(31) and (not ALU_Result(31))) or ((not A(31)) and (not B(31)) and ALU_Result(31));

    sub_ov <= (A(31) and (not B(31)) and (not ALU_Result(31))) or ((not A(31)) and B(31) and ALU_Result(31));

    with ALUCntl select
        Overflow <= add_ov when "0010",
                    sub_ov when "0110",
                    'Z' when others;

    ALUout <= ALU_Result;

end Behavioral;
```

Source code: ALU

In the ALU source code, I started with initiating the libraries for the file. Next, I declared the following four input ports A, B, Carryin, and ALUCntl, along with the four output ports ALUout, Zero, Cout, and Overflow. Afterward, I initiated the signals and their respective vectors for the architecture behavior of the ALU including ALU_Result, a32, b32, c32, add_result, sub_result, add_ov, and sub_ov. After declaring the signals, I began to assigned each port with their respective signals, then using the appropriate signals to create another signal such addition and subtraction, and finally, I combined all the signals to the output ALU_result signal. Aside

from the primary signals, I also designed the Zero, Carryout, and Overflow when the program is computing the arithmetic aspect of the lab's given testcases.

ALU_tb.vhd

Z:/Khoa Vu - Lab3/Khoa Vu - Lab3.srcs/sim_1/new/ALU_tb.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ALU_tb is
--  Port ( );
end ALU_tb;

architecture Behavioral of ALU_tb is
    signal A,B : std_logic_vector (31 downto 0);
    signal Carryin : std_logic;
    signal ALUCntl : std_logic_vector ( 3 downto 0);
    signal ALUout : std_logic_vector (31 downto 0);
    signal Zero : std_logic;
    signal Cout : std_logic;
    signal Overflow : std_logic;

    component design_1_wrapper is
    port (
        A,B : in STD_LOGIC_VECTOR (31 downto 0);
        Carryin : in STD_LOGIC;
        ALUCntl : in STD_LOGIC_VECTOR (3 downto 0);
        ALUout : out STD_LOGIC_VECTOR (31 downto 0);
        Zero : out STD_LOGIC;
        Cout : out STD_LOGIC;
        Overflow : out STD_LOGIC
        );
    end component design_1_wrapper;

begin
    design_1_i : design_1_wrapper
    port map(
        A => A,
        B => B,
        Carryin => Carryin,
        ALUCntl => ALUCntl,

        Zero => Zero,
        Cout => Cout,
        Overflow => Overflow,
        ALUout => ALUout
        );
process
```

Source code: ALU_tb (testbench)

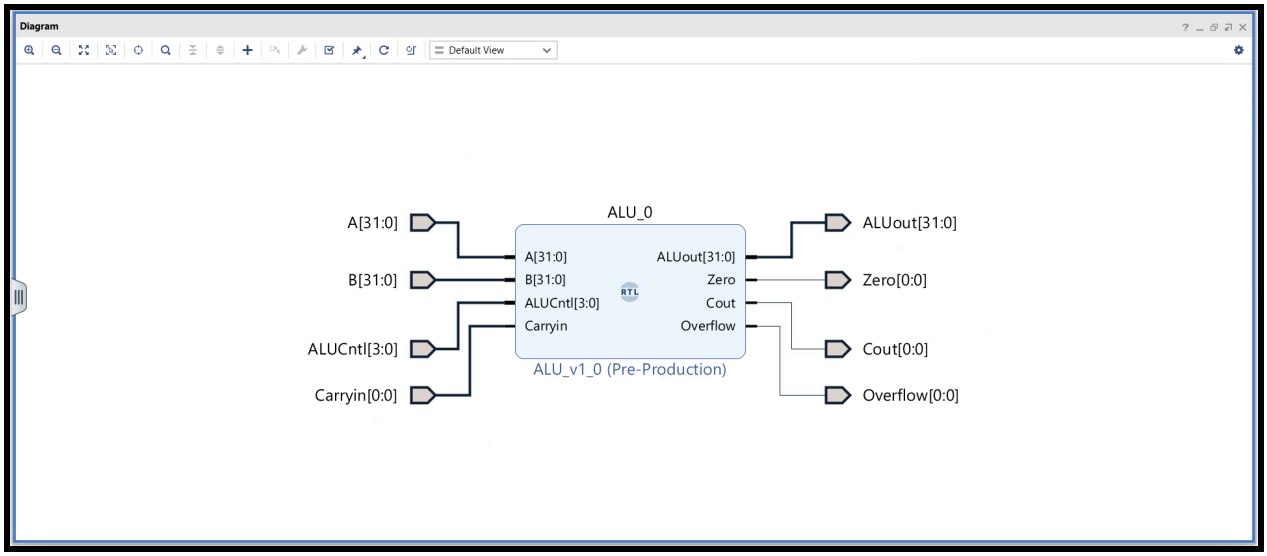
Once the ALU is done, I created a testbench source code that allowed the wrapper to be instatiated to which allowing the simulation to run and generate the necessary waveforms later on. At its core, the testbench consist of entity without any IO ports, design instantiated as component, clock input, and various stimulus inputs. With this in mind, I started with initiating the libraries for the file. Then, I wrote my architecture body with all the signals and their respective vectors. Afterward, I made the component block and wrote all of my ports that separated into four inputs and outputs that connected back to the ALU. Once the ports and signals are initiated, I assigned each port with their respective signal. In the end, when I have all

the appropriate codes for the testbench, I moved onto implementing the test cases as mentioned in the instruction.

```
begin
--test case 1
    A <= x"ffffffff";
    B <= x"00000000";
    Carryin <= '0';
    ALUCntl <= "0000";
    wait for 70ns;

--test case 2
    A <= x"98989898";
    B <= x"89898989";
    Carryin <= '0';
    ALUCntl <= "0001";
    wait for 70ns;

--test case 3 (Start here
    A <= x"01010101";
    B <= x"10101010";
    Carryin <= '0';
    ALUCntl <= "0011";
    wait for 70ns;

--test case 4
    A <= x"00000001";
    B <= x"FFFFFFFF";
    Carryin <= '0';
    ALUCntl <= "0010";
    wait for 70ns;

--test case 5
    A <= x"6389754F";
    B <= x"AD5624E6";
    Carryin <= '0';
    ALUCntl <= "0010";
    wait for 70ns;

--test case 6
    A <= x"00000001";
    B <= x"FFFFFFFF";
    Carryin <= '1';
    ALUCntl <= "0010";
    wait for 70ns;

--test case 7
    A <= x"6389754F";
    B <= x"AD5624E6";
    Carryin <= '1';
    ALUCntl <= "0010";
    wait for 70ns;

--test case 8
    A <= x"FFFFFFFF";
    B <= x"FFFFFFFF";
    Carryin <= '1';
    ALUCntl <= "0010";
    wait for 70ns;

--test case 9
    A <= x"00000000";
    B <= x"00000001";
    Carryin <= '0';
    ALUCntl <= "0110";
    wait for 70ns;

--test case 10
    A <= x"F9684783";
    B <= x"F998D562";
    Carryin <= '0';
    ALUCntl <= "0110";
    wait for 70ns;

--test case 11
    A <= x"9ABCDEDF";
    B <= x"9ABCDEFD";
    Carryin <= '0';
    ALUCntl <= "1100";
    wait for 70ns;

--test case 12
    A <= x"89BCDE34";
    B <= x"C53BD687";
    Carryin <= '1';
    ALUCntl <= "0010";
    wait for 70ns;
end process;

end Behavioral;
```

Source code: Test Cases (in testbench)

Above are the test cases that has been implemented to assess the function of my ALU. The test cases consisted a wide range of numbers that are being added and subtracted, and if necessary, the calculation have a carrying which accompanies the calculation.

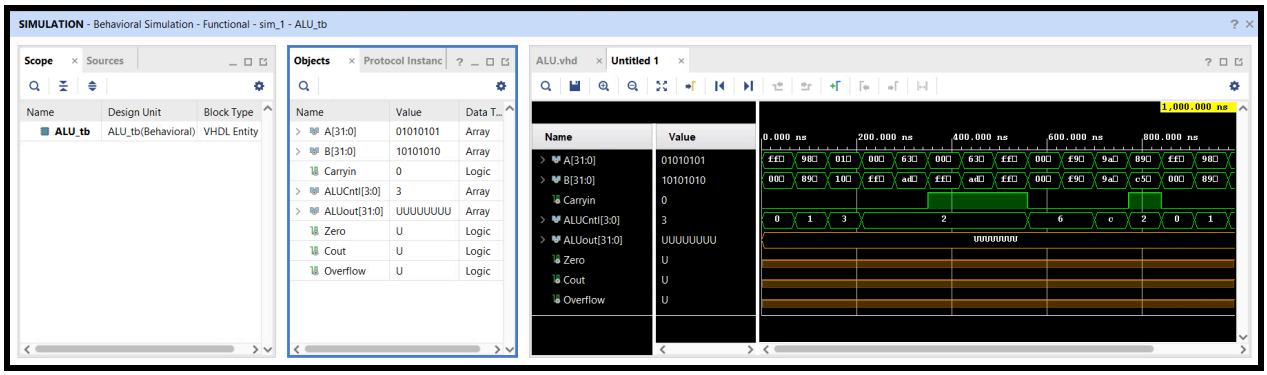


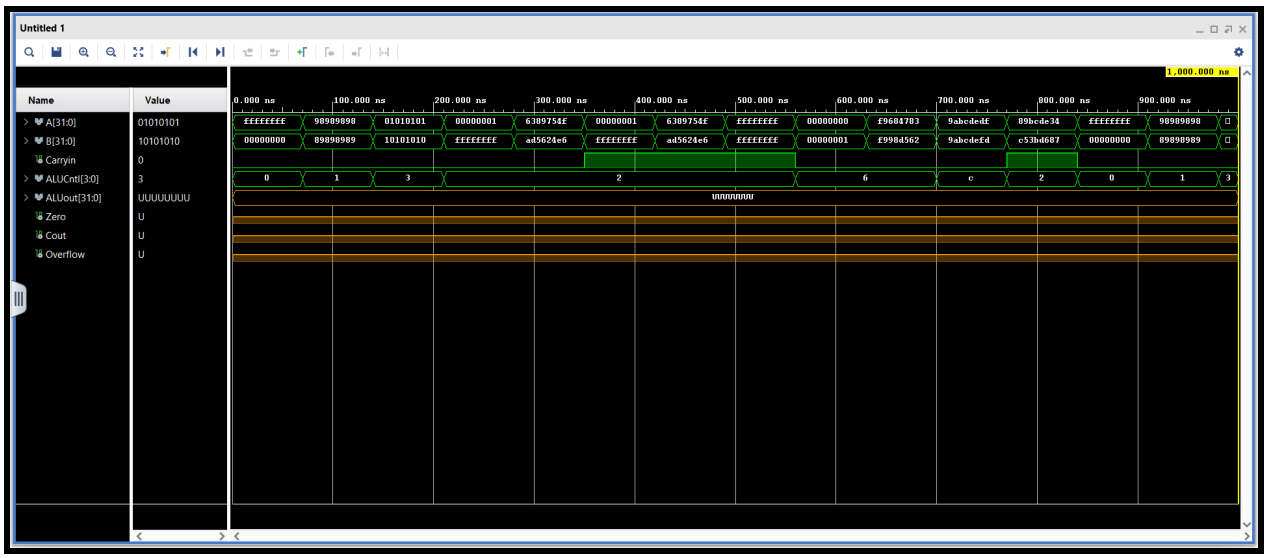
Block Design

The diagram above showcases the block design of the Arithmetic and Logic Unit. For implementation, I constructed and connected the following one full adders with both ends connecting to eight ports–four inputs/output s. Based off the ALU source code, the four input ports consisting of A, B, ALUCntl, and Carrying–each with their own vectors– are served as the inputs and connect to full adder. On the other hand, the four output ports containing ALUout, Zero, Cout, and Overflow–each with their own vectors–are served as the outputs and connect to the full adder. In details, the A and B ports take in value to be calculated meanwhile ALUCnt and Carryin accompany the calculation; moreover, the outcomes are then outputted to the ALUout port and the Zero and Cout and if necessary, the Overflow. In essence, there are buses that transfer data to registers, memory and ALU. As a result, multiplexers select inputs and decoders route data to related components, and registers temporarily store data.

<u>Analysis:</u>

Once the lab design has been completed, I ran the simulation on the program and below are the results of my design.





Simulation Waveform

According to the snapshots of the waveform, the simulation was generated from a series of inputs that are the values from the given test cases. With each test cases are design to carry out different arithmetic operations, the ALUout values are varies among the series of test cases depending on their respective inputs. In the end, the above simulation waveform are the results of the program.

Table 1: ALU Control Lines and corresponding ALU Operations

ALUCntl	Function
0000	AND
0001	OR
0011	XOR
0010	ADD
0110	SUBTRACT
1100	NOR

Table 2: Test Vectors for testing the design

#	AHex	BHex	Carryin	ALUCntl	ALUthex	Zero	Overflow	Carryout
1	FFFFFFFF	00000000	-	0000	00000000	1	0	0
2	98989898	89898989	-	0001	99999999	0	0	0
3	01010101	10101010	-	0011	11111111	0	0	0
4	00000001	FFFFFFFF	0	0010	00000000	1	0	1
5	6389754F	AD5624E6	0	0010	10DF9A35	0	0	1
6	00000001	FFFFFFFF	1	0010	00000001	0	0	1
7	6389754F	AD5624E6	1	0010	10DF9A36	0	0	1
8	FFFFFFFF	FFFFFFFF	1	0010	FFFFFFFF	0	0	1
9	00000000	00000001	-	0110	FFFFFFFF	0	0	1
10	F9684783	F998D562	-	0110	FFCF7221	0	0	1
11	9ABCDEDF	9ABCDEFD	-	1100	65432100	0	0	0
12	89BCDE34	C53BD687	1	0010	4EF8B4BC	0	1	1

Test Case for Table Calculations

Test Case 1: ALUCntl → A and B

AHex	BHex	Carryin	ALUCntl	ALUthex	Zero	Overflow	Carryout
FFFFFFFF	00000000	-	0000	00000000	1	0	0

ALUCntl = 0000 that is AND operation from the Table-1

A: FFFFFFFF

A: 1111 1111 1111 1111 1111 1111 1111 1111

B: 00000000

B: 0000 0000 0000 0000 0000 0000 0000 0000

ALUout: 0000 0000 0000 0000 0000 0000 0000 0000 Or 0x00000000

Zero: 1 because ALUout is 0

Carryout: 0 because it’s a logical operation. So, it’s always 0. Overflow: 0 because it's a logical operation. So, it’s always 0. No arithmetic performed.

Test Case 2: ALUCntl → A and B

AHex	BHex	Carryin	ALUCntl	ALUout*hex*	Zero	Overflow	Carryout
98989898	89898989	-	0001	99999999	0	0	0

ALUCntl = 0001 that is OR operation from the Table -1

A: 98989898

A: 1001 1000 1001 1000 1001 1000 1001 1000

B: 8989898989

B: 1000 1001 1000 1001 1000 1001 1000 1001

ALUout: 1001 1001 1001 1001 1001 1001 1001 1001 Or 0x99999999

Zero: 0

Carryout: 0 because it’s a logical operation. So, it’s always 0. Overflow: 0 because it's a logical operation. So, it’s always 0. No arithmetic performed.

Test Case 3: ALUCntl → A and B

AHex	**BHex**	**Carryin**	**ALUCntl**	**ALUout*hex***	**Zero**	**Overflow**	**Carryout**
01010101	10101010	-	0011	11111111	0	0	0

ALUCntl = 0011 that is XOR operation from the Table -1

A: 01010101

A: 0000 0001 0000 0001 0000 0001 0000 0001

B: 10101010

B: 0001 0000 0001 0000 0001 0000 0001 0000

ALUout: 0001 0001 0001 0001 0001 0001 0001 0001 Or 0x11111111

Zero: 0

Carryout: 0 because it’s a logical operation. So, it’s always 0. Overflow: 0 because it's a logical operation. So, it’s always 0. No arithmetic performed.

Test Case 4: ALUCntl → A and B

AHex	BHex	Carryin	ALUCntl	ALUout*hex*	Zero	Overflow	Carryout
00000001	FFFFFFFF	0	0010	00000000	1	0	1

ALUCntl = 0010 that is ADD operation from the Table -1

A: 00000001

A: 0000 0000 0000 0000 0000 0000 0000 0001

B: FFFFFFFF

B: 1111 1111 1111 1111 1111 1111 1111 1111

Carryin: 0

ALUout: 0000 0000 0000 0000 0000 0000 0000 0000 Or 0x00000000

Zero: 1 because ALUout is 0

Carryout: 1 because it’s a logical operation. So, it’s always 1. Overflow: 0 because it's a logical operation. So, it’s always 0. Arithmetic performed.

Test Case 5: ALUCntl → A and B

AHex	BHex	Carryin	ALUCntl	ALUout*hex*	Zero	Overflow	Carryout
6389754F	AD5624E6	0	0010	10DF9A35	0	0	1

ALUCntl = 0010 that is ADD operation from the Table -1

A: 6389754F

A: 0110 0011 1000 1001 0111 0101 0100 1111

B: AD5624E6

B: 1010 1101 0101 0110 0010 0100 1110 0110

Carryin: 0

ALUout: 0001 0000 1101 1111 1001 1010 0011 0101 Or 0x10DF9A35

Zero: 0

Carryout: 1 because it’s a logical operation. So, it’s always 1. Overflow: 0 because it's a logical operation. So, it’s always 0. Arithmetic performed.

Test Case 6: ALUCntl → A and B

AHex	**BHex**	**Carryin**	**ALUCntl**	**ALUout*hex***	**Zero**	**Overflow**	**Carryout**
00000001	FFFFFFFF	1	0010	00000001	0	0	1

ALUCntl = 0010 that is ADD operation from the Table -1

A: 0000001

A: 0000 0000 0000 0000 0000 0000 0000 0001

B: FFFFFFFF

B: 1111 1111 1111 1111 1111 1111 1111 1111

Carryin: 1

ALUout: 0000 0000 0000 0000 0000 0000 0000 0001 Or 0x00000001

Zero: 0

Carryout: 1 because it’s a logical operation. So, it’s always 1. Overflow: 0 because it's a logical operation. So, it’s always 0. Arithmetic performed.

Test Case 7: ALUCntl → A and B

AHex	**BHex**	**Carryin**	**ALUCntl**	**ALUout*hex***	**Zero**	**Overflow**	**Carryout**
6389754F	AD5624E6	1	0010	10DF9A36	0	0	1

ALUCntl = 0010 that is ADD operation from the Table -1

A: 6389754F

A: 0110 0011 1000 1001 0111 0101 0100 1111

B: AD5624E6

B: 1010 1101 0101 0110 0010 0100 1110 0110

Carryin: 1

ALUout: 0001 0000 1101 1111 1001 1010 0011 0110 Or 0x10DF9A36

Zero: 0

Carryout: 1 because it’s a logical operation. So, it’s always 1. Overflow: 0 because it's a logical operation. So, it’s always 0. Arithmetic performed.

Test Case 8: ALUCntl → A and B

AHex	BHex	Carryin	ALUCntl	ALUout*hex*	Zero	Overflow	Carryout
FFFFFFFF	FFFFFFFF	1	0010	FFFFFFFF	0	0	1

ALUCntl = 0010 that is ADD operation from the Table -1

A: FFFFFFFF

A: 1111 1111 1111 1111 1111 1111 1111 1111

B: FFFFFFFF

B: 1111 1111 1111 1111 1111 1111 1111 1111

Carryin: 1

ALUout: 1111 1111 1111 1111 1111 1111 1111 1111 Or 0xFFFFFFFF

Zero: 0

Carryout: 1 because it’s a logical operation. So, it’s always 1. Overflow: 0 because it's a logical operation. So, it’s always 0. Arithmetic performed.

Test Case 9: ALUCntl → A and B

AHex	BHex	Carryin	ALUCntl	ALUout*hex*	Zero	Overflow	Carryout
00000000	00000001	-	0110	FFFFFFFF	0	0	1

ALUCntl = 0110 that is SUBTRACT operation from the Table -1

A: 00000000

A: 0000 0000 0000 0000 0000 0000 0000 0000

B: 00000001

B: 0000 0000 0000 0000 0000 0000 0000 0001

ALUout: 1111 1111 1111 1111 1111 1111 1111 1111 Or 0xFFFFFFFF

Zero: 0

Carryout: 1 because it’s a logical operation. So, it’s always 1. Overflow: 0 because it's a logical operation. So, it’s always 0. Arithmetic performed.

Test Case 10: ALUCntl → A and B

AHex	**BHex**	**Carryin**	**ALUCntl**	**ALUout*hex***	**Zero**	**Overflow**	**Carryout**
F9684783	F998D562	-	0110	FFCF7221	0	0	1

ALUCntl = 0110 that is SUBTRACT operation from the Table -1

A: F9684783

A: 1111 1001 0110 1000 0100 0111 1000 0011

B: F998D562

B: 1111 1001 1001 1000 1101 0101 0110 0010

ALUout: 1111 1111 1100 1111 0111 0010 0010 0001 Or 0xFFCF7221

Zero: 0

Carryout: 1 because it’s a logical operation. So, it’s always 1. Overflow: 0 because it's a logical operation. So, it’s always 0. Arithmetic performed.

Test Case 11: ALUCntl → A and B

AHex	BHex	Carryin	ALUCntl	ALUout*hex*	Zero	Overflow	Carryout
9ABCDEDF	9ABCDEFD	-	1100	65432100	0	0	0

ALUCntl = 1100 that is NOR operation from the Table -1

A: 9ABCDEDF

A: 1001 1010 1011 1100 1101 1110 1101 1111

B: 9ABCDEFD

B: 1001 1010 1011 1100 1101 1110 1111 1101

ALUout: 0110 0101 0100 0011 0010 0001 0000 0000 Or 0x65432100

Zero: 0

Carryout: 0 because it’s a logical operation. So, it’s always 0. Overflow: 0 because it's a logical operation. So, it’s always 0. No arithmetic performed.

Test Case 12: ALUCntl → A and B

AHex	BHex	Carryin	ALUCntl	ALUout*hex*	Zero	Overflow	Carryout
89BCDE34	C53BD687	1	0010	4EF8B4BC	0	1	1

ALUCntl = 0010 that is ADD operation from the Table -1

A: 89BCDE34

A: 1000 1001 1011 1100 1101 1110 0011 0100

B: C53BD687

B: 1100 0101 0011 1011 1101 0110 1000 0111

Carryin: 1

ALUout: 0100 1110 1111 1000 1011 0100 1011 1100 Or 0x4EF8B4BC

Zero: 0

Carryout: 1 because it’s a logical operation. So, it’s always 1. Overflow: 1 because it's a logical operation. So, it’s always 1. Arithmetic performed.

Conclusion:

In conclusion, I’ve learned more about the developing process of a VHDL program and a simple testbench that verifies the functionality of an Arithmetic and Logic Unit (ALU) component. At first, I wrote and completing the necessary codes for the ALU. Then, I designed and created a block design using eight ports and one full adder. Henceforth, I worked on the testbench and once that is done, I instantiated the wrapper around the program and ran the simulation. Overall, the ALU calculated and generated outcomes that matches the manual calculation of the test cases.